



Computer Networks

Week # 07

M.Nadeem Akhtar
CS & IT Department
The University of Lahore

Email: nadeem.akhtar@cs.uol.edu.pk

URL- <https://sites.google.com/view/nadeemuolcscn/home/course-material>

ACKNOWLEDGMENTS

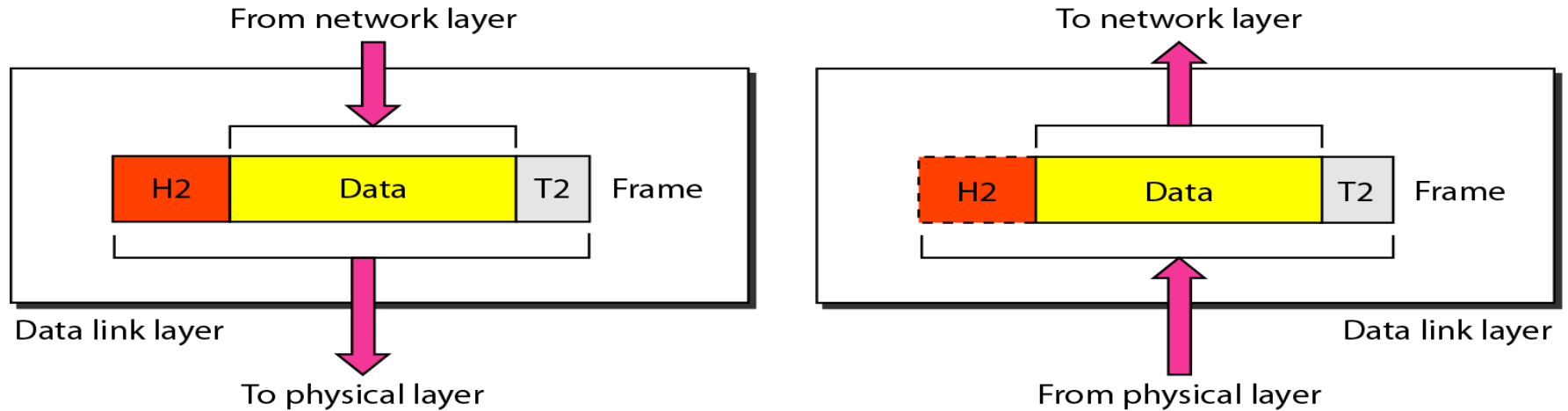
Mostly adopted from lecture slides by Behrouz A. Forouzan.

Week 7: Course Plan

Data Link Layer

- Types of Framing
 - Byte count
 - character-based framing
 - Bit-oriented framing
- Flow control
 - ARQ Protocols
- Error detection and correction issues
- Error detection codes
- Error correction codes

Data Link Layer



The data link layer is responsible for moving frames from one hop (node) to the next

Functions of the Data Link Layer

(local focus)

Specific things that DLL does in order to accomplish the general functions of handling errors and regulating data flow

Framing: divides the stream of bits received from the network layer into manageable data units called frames

Flow Control (imposes a flow control mechanism to prevent sender from “swamping” or overwhelming the receiver which is normally built into the protocol)

Error Control: adds reliability to the physical layer by adding mechanisms to detect and retransmit damaged, duplicate, or lost frames. Both detection and correction schemes may be used

Media access control determines which device has control over the link at any given time when two or more devices are connected to the same link

Addressing adds a header to the frame to define the addresses of the sender and receiver of the frame

Framing

- The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another
- Deciding, at the receiving end, where frames start and stop and filling idle periods

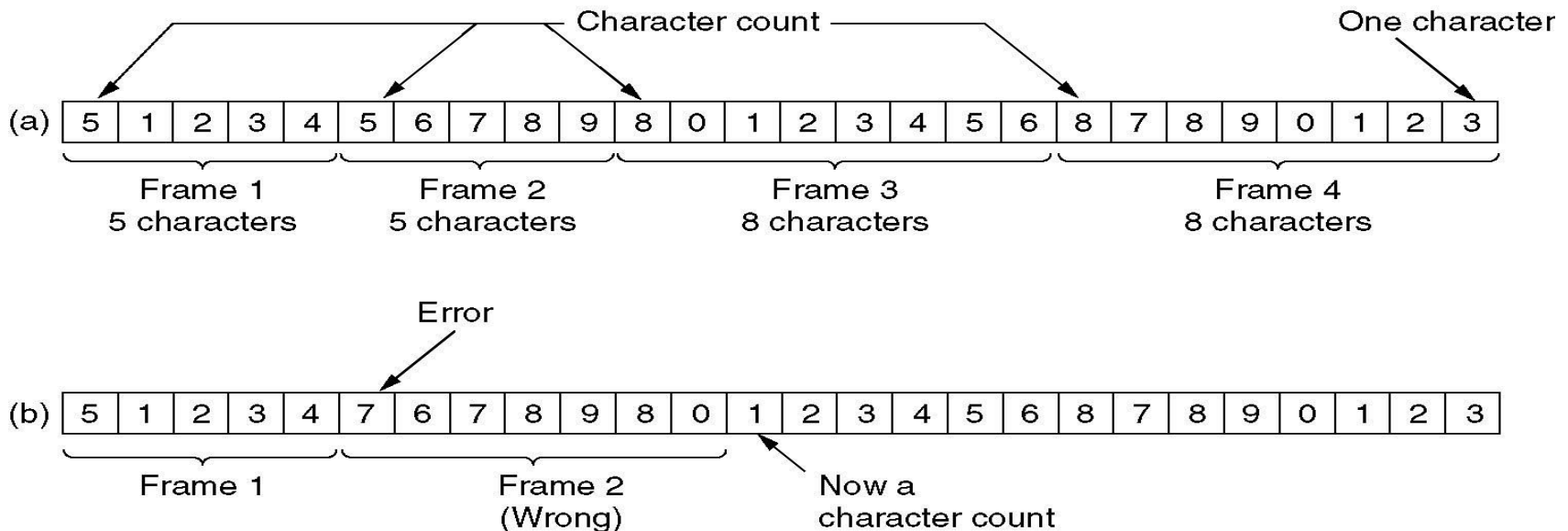
Types of framing

- byte count
- character-based framing
- bit-oriented framing

Framing: Byte (length) count

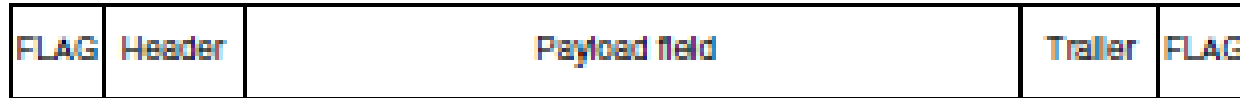
- Header contains length of frame
- **Problems**
 - count may become corrupted
 - receiver has no way to tell where next frame starts

Example:



Framing: character-based framing

- special characters (flag bytes) used with byte stuffing



- Two consecutive flag bytes indicate the end of one frame and the start of the next
- Thus, if the receiver ever loses synchronization
 - it can just search for two flag bytes to find the end of the current frame and the start of the next frame
- If FLAG byte occurs in data itself (such as in binary data: transmission of photographs or songs)
 - Interfering problem
- **Insert ESC byte before each FLAG data byte (byte stuffing)**
- If ESC byte itself appears in data
 - Insert another ESC byte before ECS byte of data

Framing: character-based framing

Interfering problem

If FLAG byte occurs in data itself (such as in binary data: transmission of photographs or songs)

➤ Insert ESC byte before each FLAG data byte (**byte stuffing**)

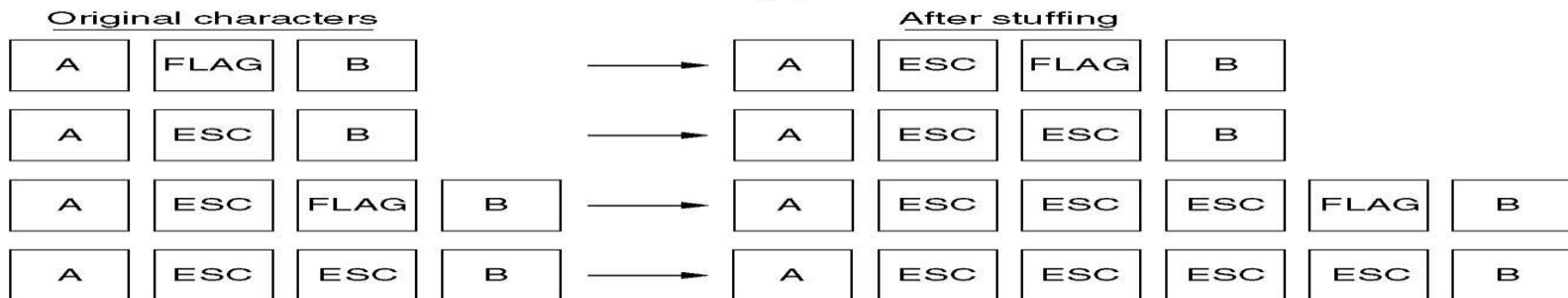
If ESC byte itself appears in data

➤ Insert another ESC byte before ECS byte of data

Byte stuffing is the process of adding 1 extra byte whenever there is a flag or escape character in the text



(a)



(b)

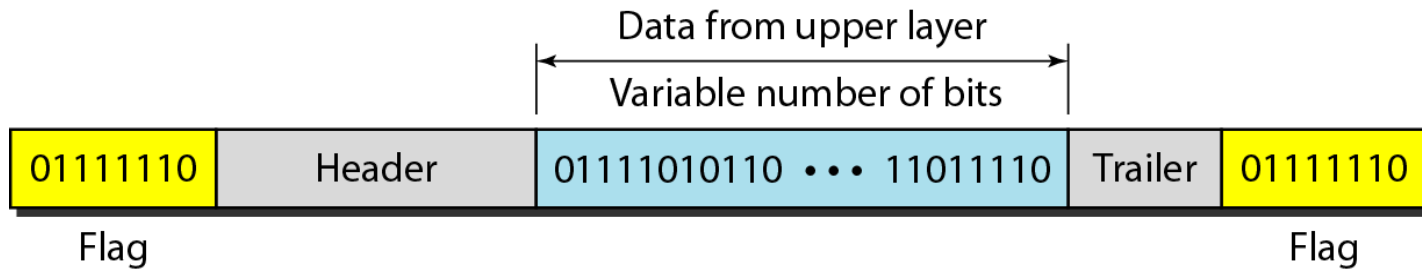
(a) A frame delimited by flag bytes

(b) Four examples of byte sequences before and after stuffing

Framing: bit-oriented framing

Special bit sequences (flag bits) used with bit stuffing

- Developed for High level Data Link Control (HDLC) protocol
- Each frame begins and ends with a special bit pattern, 01111110 or 0x7E in hexadecimal (a flag byte)



Whenever the sender's data link layer encounters five consecutive 1s in the data,

- it automatically stuffs a 0 bit into the outgoing bit stream so that the receiver does not mistake the pattern 01111110 for a flag. This process is called **Bit Stuffing**

When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit,

- it automatically de-stuffs (i.e., deletes) the 0 bit

Framing: bit-oriented framing

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0



Stuffed bits

The diagram shows three arrows pointing from the text 'Stuffed bits' to the stuffed 0 bits in the transmitted frame (b). The first arrow points to the 10th bit (0), the second arrow points to the 15th bit (0), and the third arrow points to the 21st bit (0). These stuffed 0s ensure that no 0010 bit sequence appears in the frame.

(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Bit stuffing

(a) The original data

(b) The data as they appear on the line

(c) The data as they are stored in receiver's memory after de-stuffing

Flow Control

- A technique for assuring that a transmitting entity does not overwhelm a receiving entity with data (speed limit)
- The receiving entity typically allocates a data buffer of some maximum length for a transfer
 - When data are received, the receiver must do a certain amount of processing (e.g., examine the header) before passing the data to the higher-level software

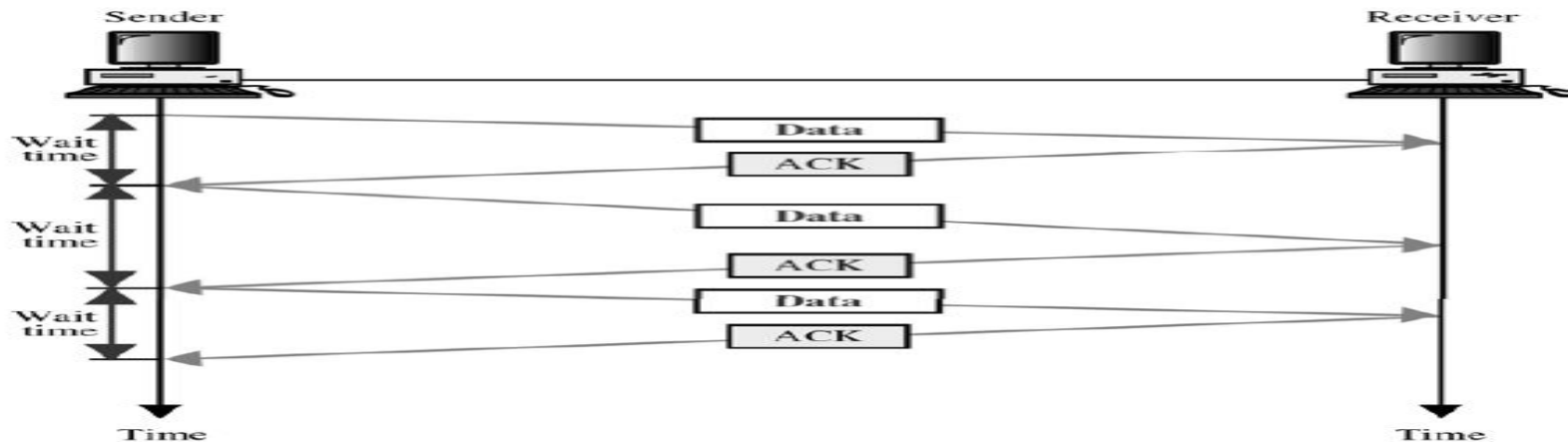
Flow Control

- **So flow control is the amount of data that can be sent before the receiving ACK**
 - It is linked to error control as it provides the ACK from receiver for correctly received frames
- In the absence of flow control
 - the receiver's buffer may **fill up and overflow** while it is processing old data
- Stop and wait and sliding window are two methods of flow control

Flow Control Methods

Stop and Wait

- sender waits for an ACK after every sent frame
- Process of sending and waiting for ACK repeats until sender sends EOT (End of Transmission) frame like dictation by officer to typist
- Simplest scheme BUT inefficient



Flow Control Methods

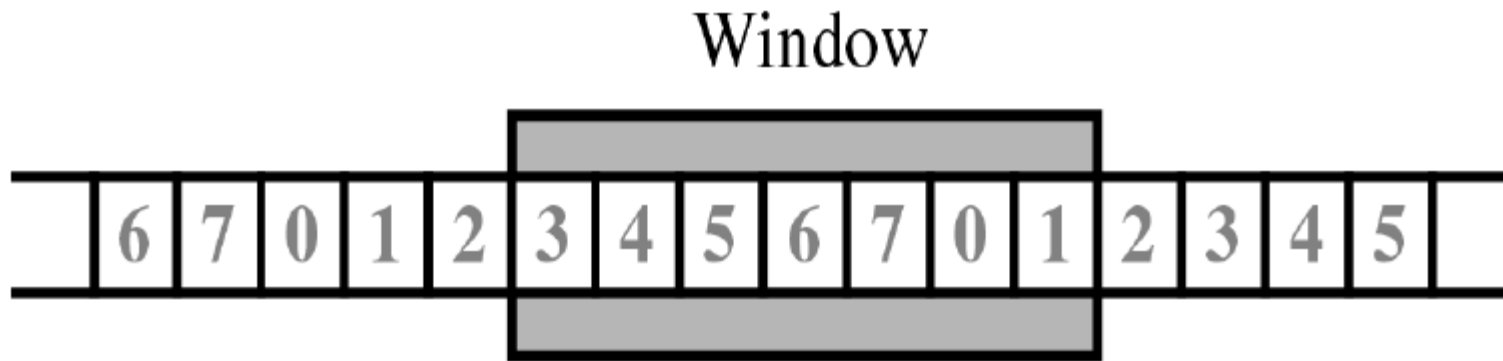
Sliding Window

- Sender can send several frames before needing an ACK
 - So link can carry several frames at once (efficient use of capacity)
 - A single ACK from receiver for multiple data frames received
 - Upper limit on frames to be sent before need of ACK
- ACK can be sent without waiting for window size to be filled up and can be transmitted even before this
- Concept of identification scheme on the basis of window size
 - To keep track of transmitted and received frames
- Frame numbering is based on modulo-n
 - from 0 to n-1 ; If n=8, frames will be numbered as
 - 0,1,2,3,4,5,6,7,0,1,2,3,4,5,6,7,0,....

Flow Control Methods

Sliding Window

- The receiver includes number of the next frame expected to be received in ACK it sends
 - E.g., when it receives a string of frames ending in frame 4,
 - The ACK is sent with number 5
- The window size is $n-1$ frames (at either end),
 - So at most $n-1$ frames may be sent before requiring an ACK



Automatic Repeat Request (ARQ) protocols

When the receiver detects an error in a packet

- it automatically requests the transmitter to resend the packet by NACK (negative ACK)
- This process is repeated until
 - the packet is error free or
 - the error continues beyond a predetermined number of transmissions
- **In ARQ, only error detection capability is provided and no attempt to correct any packets received in error is made;**
 - **instead it is requested that the packets received in error be retransmitted**
- It relies on the use of an error detection code, such as CRC

Automatic Repeat Request (ARQ) protocols

- IT uses ACKs and timeouts to achieve reliable data transmission over unreliable service
 - The packets for which the ACK is not received in predetermined time interval (timeout) are assumed to be lost and retransmitted
- ARQ may lead to variable delays which are not acceptable for real-time services

Operation of ARQ

The transmitter

- numbers the packets to be transmitted sequentially and
- maintains a timer for each packet it transmits by using numbers from a finite set

The receiver

- acknowledges, at the very least, the receipt of each successful packet by transmitting an ACK
 - Which bears the sequence number of the packet being acknowledged

ARQ Protocols

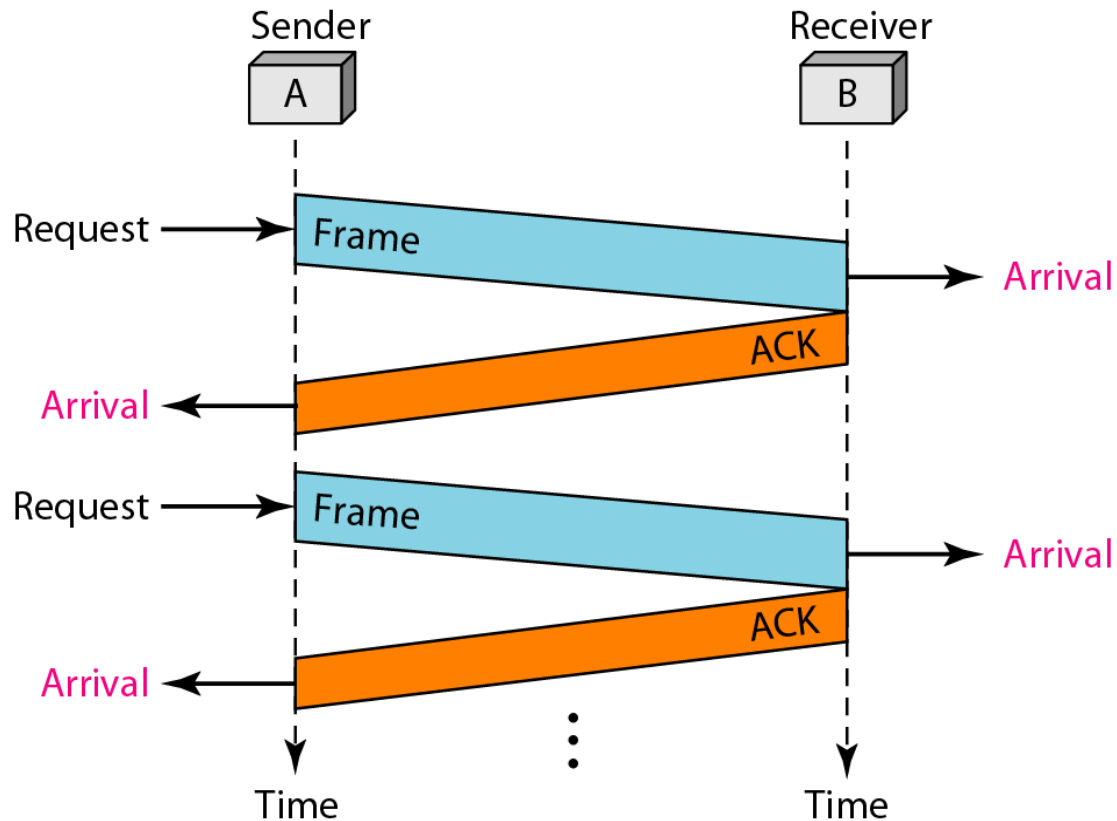
- **Stop and Wait**
- **Sliding Window**
 - **Selective repeat (reject)**
 - **Go-Back-N**

ARQ Protocols

Stop and Wait (SW)

- Type of stop-and-wait flow control
 - But extended to include retransmission of lost or damaged frames
- DLC protocol transmits a packet only when all previously transmitted packets have been successfully acknowledged
- **The transmitter after transmitting a packet waits for its ACK**
 - **Next packet is transmitted after receipt of ACK**
- If no ACK arrives within timeout
 - Packet is retransmitted
- **In SW, never more than a single packet unacknowledged at any given instant of time**
- **the maximum data transfer rate that can be supported is limited**
 - Since the transmitter does not use the available channel during time intervals it waits for an ACK

SW: Flow diagram for noiseless channel

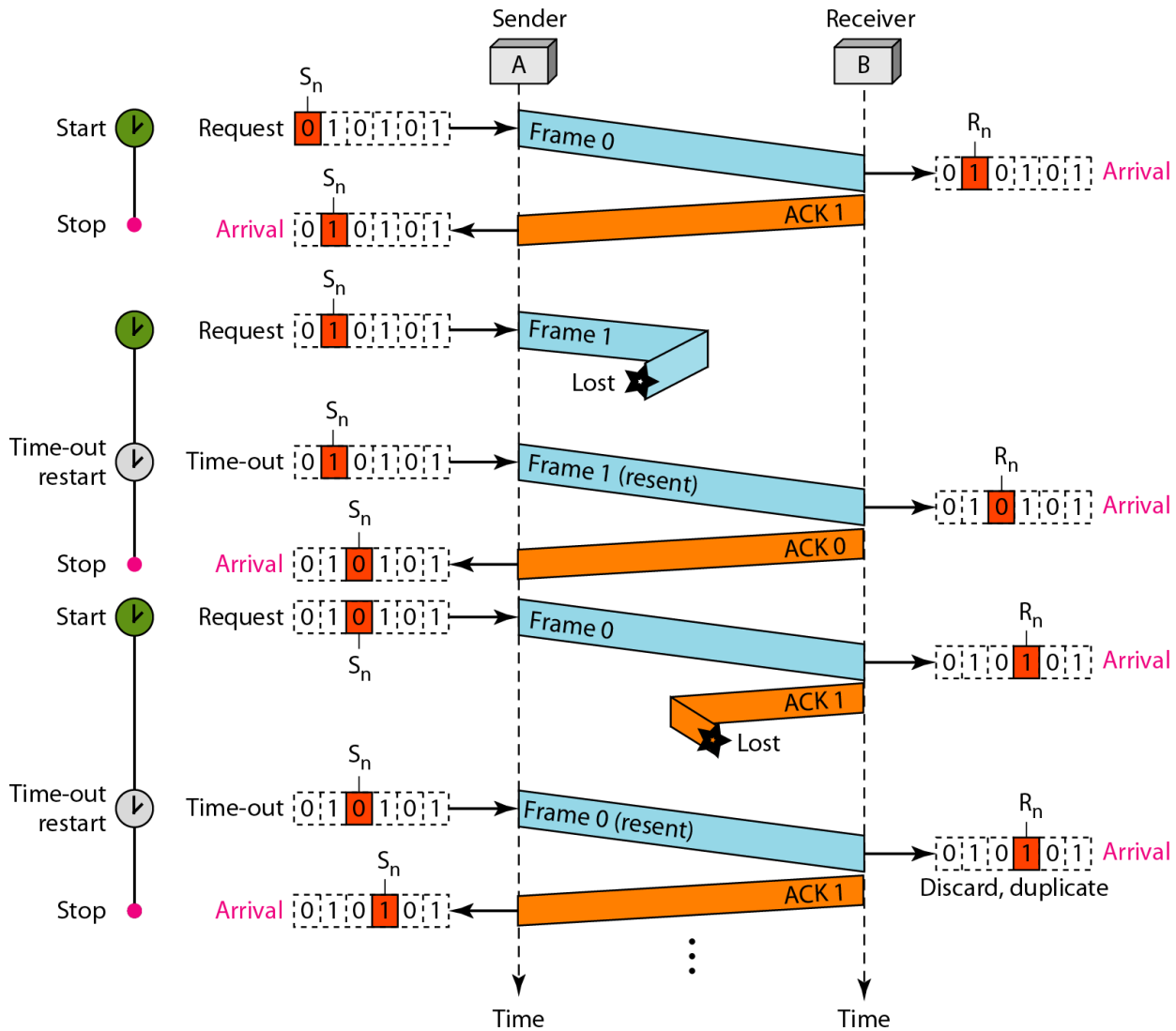


- Sender keeps a copy of the last transmitted frame till receiving of its ACK
- Both data and ACK frames are numbered 0 and 1 alternately
 - A data frame number as 0 is acknowledged by a ACK 1 frame
 - Means that receiver has received data 0 and is now expecting data 1

An example of Stop-and-Wait ARQ for a noisy channel

- ✓ Frame 0 is sent and acknowledged
- ✓ Frame 1 is lost and resent after the time-out.
- ✓ The resent frame 1 is acknowledged and the timer stops.
- ✓ Frame 0 is sent and acknowledged, but the acknowledgment is lost.
- ✓ The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged.

Flow diagram: SW ARQ for a noisy channel



ARQ Sliding Window Protocols

Features

- The sender keeps copies of transmitted frames
 - until all of them have been ACKed
- Receiver can send NACK as well
 - If data received as damaged
- Both ACK and NACK frames are numbered for identification
 - Since sliding window is a continuous TX mechanism
- ACK carries the number of next expected frame
- NACK carries the number of the damaged frame itself
 - For example, If data frames 4 and 5 are damaged, NACK 4 and NACK 5 must be sent

ARQ Sliding Window Protocols

Selective Repeat (SR)

- DLC layer transmits packets continuously when available
- The receiver ACKs for each successfully received packet
 - Bearing the sequence number of the packet being ACKed
- For non-receipt of ACK before the expiration of timeout
 - Packet is retransmitted
 - the transmitter resumes transmission of packets from where it left off
- **With SR ARQ protocol, packets are continuously transmitted**
 - **the inefficiency associated with SW is eliminated**
- **But packets can be accepted out of sequence**
 - Packets received out of sequence have to be buffered and sequenced before they can be delivered

ARQ Protocols

Go-Back-N

- packets are transmitted continuously as in SR
- But at the receiver, the DLC layer accepts packets only in the order in which they were transmitted
- Packets received out of sequence are discarded and not acknowledged
- Since the receiver accepts packets only in-sequence
 - after a timeout,
 - the transmitter retransmits the packet that timed out and
 - all packets with sequence numbers following the retransmitted packet

ARQ Protocols

Go-Back-N

- Hence, for each timeout, all packets that are yet to be acknowledged are retransmitted
- packets are transmitted continuously as in SR
- But without the need to buffer out of sequence packets
 - Hence, there is no resequencing overhead

Networks must be able to transfer data from one device to another with acceptable accuracy

Data can be corrupted during transmission

Some application can tolerate a small level of error such as random errors in audio or video transmission

But transmission of text requires very high level of accuracy

Thus, some applications require that errors be detected and corrected

In order to cope with data transmission errors

Error detection codes

Error correction codes

Error detection and correction issues

some issues related, directly or indirectly, to error detection and correction

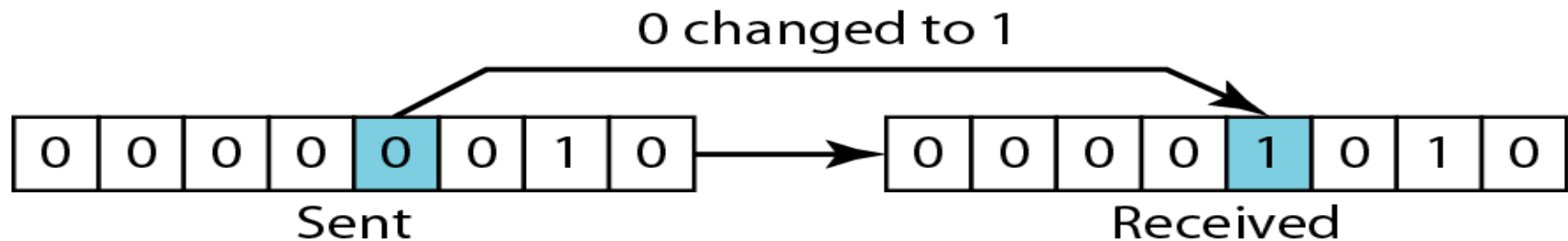
- *Types of Errors*
- *Redundancy*
- *Coding*
- *Detection versus Correction*
- *Error Correction Methods*
- *Modular Arithmetic*

➤ **NOTE: Block of data transmitted from one protocol entity to another is known as protocol data unit (PDU)**

Types of Errors

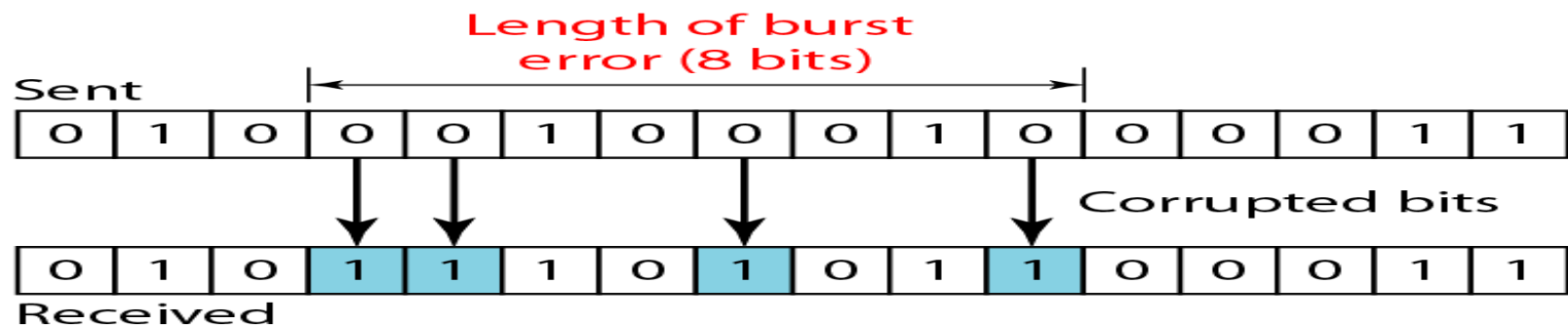
Single bit error is only 1 bit in the data unit has changed

– least likely type of error in serial data transmission



In a burst error, 2 or more bits in the data unit get changed

The length of the burst is measured from the first corrupted bit to the last corrupted bit. OR distance between the first and last errors in data block



Redundancy: Central concept in detecting /correcting errors

- ❖ Need to send extra (redundant) bits with the data
- ❖ Extra bits are added by the sender and removed by the receiver
- Presence of redundant allows receiver to detect or correct corrupted bits

Coding: Various coding schemes to achieve redundancy

- The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits
- The receiver checks the relationships between the two sets of bits to detect or correct the errors
- **In any coding scheme,** the ratio of redundant bits to the data bits and the robustness of the process are important factors

Detection Versus Correction

Error detection concerns only to see if any error has occurred

- *Simply Yes or No*

- *Even not interested in the number of errors (corrupted bits)*

- ❖ *Single bit error is same as the burst error*

Error correction: need to know the exact number of corrupted bits and their location in the message (more important)

- The number of the errors and the size of the message are important factors

- In an 8 bit data

- ❖ To correct a single bit error, need to consider eight possible error locations

- ❖ To correct two errors, need to consider 28 possibilities or combinations

- Imagine the receiver's difficulty in finding 10 errors in a data unit of 1000 bits

Error Correction Methods

➤ Forward error correction

- ❖ The receiver tries to guess the message by using redundant bits if the number of errors is small

➤ Correction by retransmission

- ❖ A technique in which the receiver detects the occurrence of an error and asks the sender to resend the message
- ❖ Resending is repeated until a message arrives error free (as per believe of the receiver)

Modular Arithmetic

- uses only a limited range of integers
- *modulo- N arithmetic*
 - ❖ define an upper limit, called a modulus N
 - ❖ then use only the integers 0 to $N - 1$, *inclusive*
 - ❖ no carry when adding two digits in a column
 - ❖ no borrow when subtracting one digit from another in a column
- **In Modulo-2 arithmetic** (*XORing of two single bits or words*)

$$0 \oplus 0 = 0 \qquad 1 \oplus 1 = 0$$

a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

	1	0	1	1	0
$\oplus$	1	1	1	0	0
	0	1	0	1	0

c. Result of XORing two patterns

Error Control Requirements

The most common techniques for error control are based on some or all of the following ingredients:

- **Error detection**
 - Receiver detects errors and discards PDUs in error
- **Positive acknowledgement**
 - Destination returns acknowledgment of successfully received, error-free PDUs
- **Retransmission** after timeout
 - Source retransmits unacknowledged PDUs after a predetermined amount of time
- **Negative acknowledgement and retransmission**
 - Destination returns negative acknowledgment to PDUs in which an error is detected
 - The source retransmits such PDUs

Error Detection Codes

- Data are transmitted as one or more contiguous sequences of bits, called frames
- Data transmission can contain errors (single bit or burst)
- Error detection codes detect the presence of an error
- **How to detect errors?**
 - ❖ If only data is transmitted, errors cannot be detected
 - ❖ Send more information with data that satisfies a special relationship
 - Add redundancy
- **Error-detecting codes are commonly used in link, network, and transport layers**

Error Detection Process

Transmitter

- For a given frame of bits , the transmitter adds additional bits that constitute an error-detecting code
- an error-detecting code (check bits) is calculated from transmitted (data) bits
- Check bits are appended to data bits

Receiver

- Separates incoming frame into data bits and check bits
- Calculates check bits from received data bits
- Compares calculated check bits against received check bits
- A detected error occurs if and only if there is a mismatch

Error Detection Process

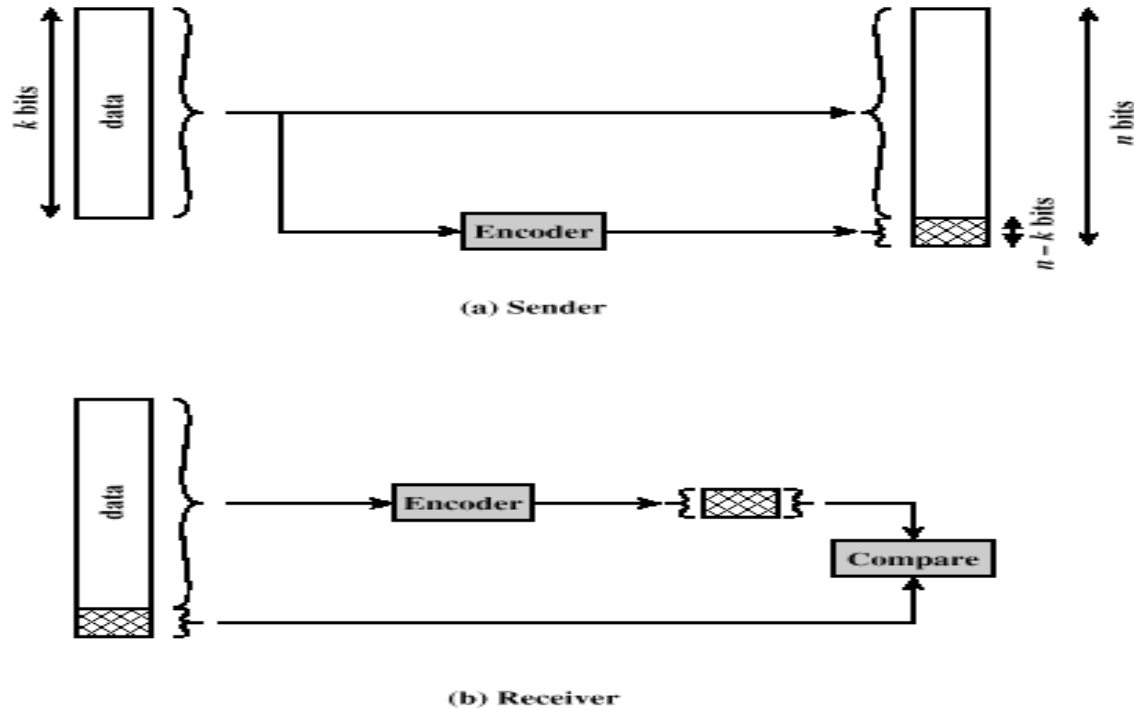


Figure 8.1 Error Detection Process

Taken from “Wireless Communications & Networks” by William Stallings

Error Detection Codes

Parity Check

- Parity bit (a single bit) appended at the end of data block
- **Even parity**
 - ❖ Added bit ensures an even number of 1s
- **Odd parity**
 - Added bit ensures an odd number of 1s
- Example, 7-bit character [1110001]
 - Even parity [1110001**0**]
 - Odd parity [1110001**1**]

Parity Check

- If the transmitter is transmitting **1110001** and using odd parity
 - ❖ It will append a **1** and transmit **11100011**
 - ❖ The receiver examines the received character and if the total number of 1s is odd
 - ❖ No error
 - ❖ If 1 bit or any odd number of bits is inverted during transmission, For example,
 - ❖ Then the receiver will detect an error
- **Performance:**
 - ❖ Detects all odd-number errors in a data block (1,3,5,...bits in error)
 - ❖ Detects NO errors that flip an even number of bits (2, 4, 6, ... bits in error)

Error Detection Codes:

Cyclic Redundancy Check (CRC)

- One of the most common and powerful error-detecting codes
- **Transmitter**
 - ❖ For a given k -bit block, transmitter generates an $(n-k)$ -bit **frame check sequence (FCS)**
 - ❖ Resulting frame consisting of n bits is exactly divisible by **predetermined number** (a pattern)
- **Receiver**
 - ❖ Divides incoming frame by **predetermined number**
 - ❖ If no remainder, assumes no error
- Procedure can be represented by
 - ❖ Modulo 2 Arithmetic
 - ❖ Polynomials

Modulo 2 Arithmetic

- Modulo 2 arithmetic is performed digit by digit on binary numbers
- Each digit is considered independently from its neighbors
- Binary addition with no carries: Exclusive-OR (XOR)
- Binary subtraction with no borrows: as the XOR operation

$$\begin{array}{r} 1111 \\ + 1010 \\ \hline 0101 \end{array}$$

$$\begin{array}{r} 1111 \\ - 0101 \\ \hline 1010 \end{array}$$

$$\begin{array}{r} 11001 \\ \times 11 \\ \hline 11001 \\ 11001 \\ \hline 101011 \end{array}$$

CRC using Modulo 2 Arithmetic

Parameters:

- **T = n -bit frame to be transmitted**
- **D = k -bit block of data; the first k bits of T**
- **F = $(n - k)$ -bit FCS; the last $(n - k)$ bits of T**
- **P = pattern of $n - k + 1$ bits; this is the predetermined divisor**
- Q = Quotient
- R = Remainder

CRC using Modulo 2 Arithmetic: Example

Given: *Pattern* (divisor) $P = 110101$ (6 bits) ----- $n-k+1$

FCS = to be calculated ($n-k = 5$ bits)

Message D (k -bit block) = 1010001101 (10 bits)

– Thus, n (total bits) = 15 (as $n-k+1 = 6$), $k = 10$ and $n-k = 5$ bits

- The message is multiplied by 2^5 , producing 101000110100000
- The product is divided by P
- The remainder is added to 2^5D to give $T = 101000110101110$
- If no errors, then receiver receives T as it is. The received frame is divided by P
- **If there is no remainder, it is assumed that there have been no errors**

$$\begin{array}{r}
 \begin{array}{c} P \rightarrow 110101 \end{array} \overline{) \begin{array}{cccccccccccc} & & & & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 0 & 1 & 1 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 0 & 1 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 1 & 1 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 0 & 1 & 1 & 0 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 0 & 0 & 1 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 0 & 1 & 1 & 1 & 0 & & & & & & & & & & \\ \hline
 \end{array}
 \end{array}$$

$Q \leftarrow 11010101110$
 $2^{n-k}D \leftarrow 101000110100000$
 $R \leftarrow 01110$

$$\begin{array}{r}
 \begin{array}{c} P \rightarrow 110101 \end{array} \overline{) \begin{array}{cccccccccccc} & & & & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 0 & 1 & 1 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 0 & 1 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 1 & 1 & 1 & 0 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 0 & 1 & 1 & 1 & 1 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 1 & 1 & 0 & 1 & 0 & 1 & & & & & & & & & \\ \hline 0 & & & & & & & & & & & & & & \\ \hline
 \end{array}
 \end{array}$$

$Q \leftarrow 11010101110$
 $T \leftarrow 101000110101110$
 $R \leftarrow 0$

CRC using Modulo 2 Arithmetic: Example

Given: *Pattern* (diviser) $P = 11001$ (5 bits) ----- $n-k+1$

FCS = to be calculated ($n-k = 4$ bits)

Message D (k -bit block) = 110011 (6 bits)

– Thus, n (total bits) = 10 (as $n-k+1 = 5$), $k = 6$ and $n-k = 4$ bits

➤ Transmitted block $T = 1100111001$

$$\begin{array}{r}
 100001 \\
 11001 \overline{)1100110000} \\
 \underline{11001} \\
 10000 \\
 \underline{11001} \\
 1001 = R(x)
 \end{array}$$

Send the block 110011 1001

At Receiver

$$\begin{array}{r}
 11001 \overline{)1100111001} \\
 \underline{11001} \\
 11001 \\
 \underline{11001} \\
 00000
 \end{array}$$

No remainder
→ Accept

CRC using Polynomials

- All values expressed as polynomials
 - ❖ Dummy variable X with binary coefficients
 - ❖ The coefficients correspond to the bits in the binary number
- For $D = 1010001101$ ----- $D(X) = X^9 + X^7 + X^3 + X^2 + 1$
- For $P = 110101$ ----- $P(X) = X^5 + X^4 + X^2 + 1$

CRC using Polynomials: Example

➤ Taking previous example

➤ For $D = 1010001101$ ----- $D(X) = X^9 + X^7 + X^3 + X^2 + 1$

➤ For $P = 110101$ ----- $P(X) = X^5 + X^4 + X^2 + 1$

➤ End up with $R = 01110$ ----- $R(X) = X^3 + X^2 + X$

$$\begin{array}{r}
 P(X) \rightarrow X^5 + X^4 + X^2 + 1 \quad \begin{array}{l} \overline{X^9 + X^8 + X^6 + X^4 + X^2 + X} \\ \overline{X^{14} \phantom{+ X^{13}} + X^{12} \phantom{+ X^{11}} + X^8 + X^7 + X^5} \end{array} \quad \begin{array}{l} \leftarrow Q(X) \\ \leftarrow X^5 D(X) \end{array} \\
 \hline
 \begin{array}{r} X^{14} + X^{13} + \phantom{X^{12}} + X^{11} + \phantom{X^{10}} + X^9 \\ \hline X^{13} + X^{12} + X^{11} + \phantom{X^{10}} + X^9 + X^8 \\ \hline X^{13} + X^{12} + \phantom{X^{11}} + X^{10} + + X^8 \\ \hline X^{11} + X^{10} + X^9 + + X^7 \end{array} \\
 \hline
 \begin{array}{r} X^{11} + X^{10} + + X^8 + + X^6 \\ \hline X^9 + X^8 + X^7 + X^6 + X^5 \\ \hline X^9 + X^8 + + X^6 + + X^4 \\ \hline X^7 + + X^5 + X^4 \end{array} \\
 \hline
 \begin{array}{r} X^7 + + X^4 + + X^2 \\ \hline X^6 + + + X^2 \\ \hline X^6 + X^5 + + X^3 + + X \\ \hline X^3 + X^2 + X \leftarrow R(X) \end{array}
 \end{array}$$

Error Correction Codes

- **Error detection** is found a useful technique in data link control protocols and in transport protocols (TCP)
- Error detection requires **retransmission** (using Automatic Repeat reQuest)
- **Detection** inadequate for **wireless applications**
 - **wireless** links are notoriously **noisy** and error prone when compared to optical fibers
 - **Bit error rate** on wireless link can be high, results in a large number of retransmissions
 - **Long propagation delay** compared to transmission time
- Without **error-correcting codes**, it would be hard to get anything through

We need error control mechanisms to detect and correct errors that occur in the transmission of PDUs

Forward error correction codes (FEC)

- Designed to detect and correct errors
- **Widely used form of error correction code**
 - Block error correction codes
- Follow the same general layout as in error detection codes
 - Take as input ***k-bit*** block, add ***r = n-k*** bits to produce **n** bit-block

Forward Error Correction Process

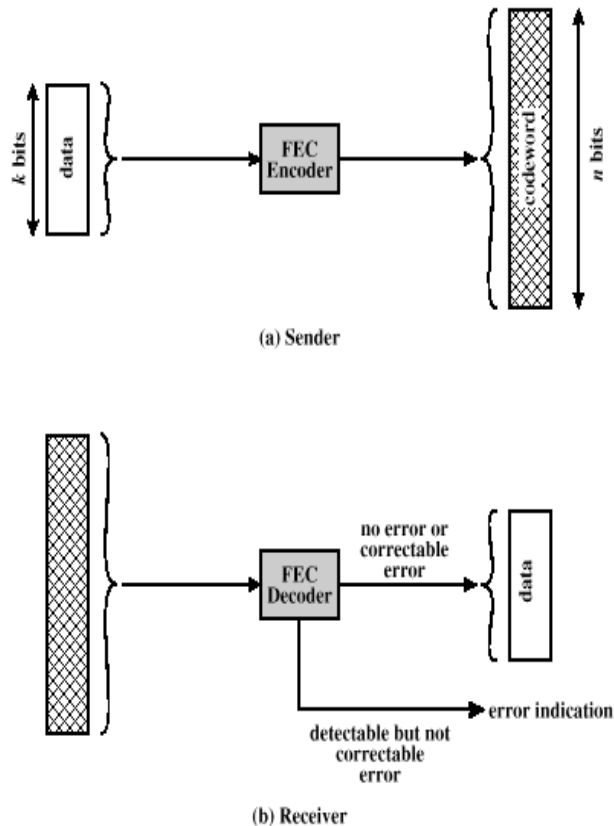


Figure 8.5 Forward Error Correction Process

FEC Decoder Outcomes

When a block is passed through FEC, possible outcomes

- **No errors present**
 - Input to the FEC decoder matches original codeword
 - Decoder produces the original data block as output
- **Decoder detects and corrects** bit errors for certain error patterns
- **Decoder detects but cannot correct** bit errors for certain error patterns
 - Decoder simply reports uncorrectable error
- **Decoder detects no bit errors** (for rare error patterns), though errors are present

Hamming Code Process

- Encoding: k data bits + $(n - k)$ check bits
- Decoding: compares received $(n - k)$ bits with calculated $(n - k)$ bits using XOR
 - Resulting $(n - k)$ bits called ***syndrome word***
 - Syndrome range is between **0 and $2^{(n-k)}-1$**
 - Each bit of syndrome indicates **a match (0) or conflict (1) in that bit position**

Hamming Code

- number of bits needed to correct a single bit error in a word containing k data bits

$$2^{(n-k)} - 1 \geq k + (n - k) = n$$

Table 8.1 Hamming Code Requirements

Data Bits	Single-Error Correction	
	Check Bits	% Increase
8	4	50
16	5	31.25
32	6	18.75
64	7	10.94
128	8	6.25
256	9	3.52

Hamming Code: Example

- Consider 8 bit data block 00111001
- we need 4 check bits
- **Transmitted Block**
- Check bits (hamming code) are to be added at positions 1,2,4,8,...
 - By taking XOR of data bits having value 1 (here, four data bits have value 1). Here, we get 0111 Hamming code
 - **So transmitted block is: 001101001111**

Bit Position	12	11	10	9	8	7	6	5	4	3	2	1
Position Number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data Bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check Bit					C8				C4		C2	C1
Transmitted Block	0	0	1	1	0	1	0	0	1	1	1	1
Codes			1010	1001		0111				0011		

(b) Check bit calculation prior to transmission

Position	Code
10	1010
9	1001
7	0111
3	0011
XOR = C8 C4 C2 C1	0111

Hamming Code: Example

- Now suppose Data bit 3 (at position 6) changes from 0 to 1
 - receiver receives the block as **001101101111**
 - Hamming code is still the same
 - Receiver performs XOR of hamming code and all of the bit position values for non-zero data bits with a result of 0110 (indicates there is error at in bit position 6)

Bit Position	12	11	10	9	8	7	6	5	4	3	2	1
Position Number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data Bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check Bit					C8				C4		C2	C1
Received Block	0	0	1	1	0	1	1	0	1	1	1	1
Codes			1010	1001		0111	0110			0011		

(d) Check bit calculation after reception

Position	Code
Hamming	0111
10	1010
9	1001
7	0111
6	0110
3	0011
XOR = syndrome	0110

Syndrome characteristics

- **If the syndrome contains all 0s, no error has been detected**
- If the syndrome contains one and only one bit set to 1
 - **then an error has occurred in one of the check bits**
 - No correction is needed
- If the syndrome contains more than one bit set to 1
 - then the numerical value of the syndrome indicates the **position of the data bit in error**
 - **This data bit is inverted for correction**